

pability of LLMs to successfully complete various tasks. (2) *Conversation Capability*. LLMs can usually generate high-quality conversations. This capability enables agents to behave more like humans. In the previous work, many agents take actions based on the strong conversation capability of LLMs. For example, in ChatDev [18], different agents can discuss the software development process and reflect on their own behaviors. In RLP [30], the agent can communicate with the listeners based on their potential feedback on the agent's utterance. (3) *Common Sense Understanding Capability*. Another important capability of LLMs is that they can well comprehend human common sense. Based on this capability, many agents can simulate human daily life and make human-like decisions. For example, in Generative Agent [20], the agent can accurately understand its current state, the surrounding environment, and summarize high-level ideas based on basic observations. Without the common sense understanding capability of LLMs, these behaviors cannot be reliably simulated. Similar conclusions may also apply to RecAgent [21] and S3 [78], where the agents focus on simulating user social behaviors.

Action Impact: Action impact refers to the consequences of an agent's actions. While the range of possible impacts is vast, we highlight a few key examples for clarity: (1) *Changing Environments*. Agents can directly alter environment states by actions, such as moving their positions, collecting items, constructing buildings, etc. For instance, in GITM [16] and Voyager [38], the environments are changed by the actions of the agents in their task completion process. Specifically, when an agent collects resources—such as harvesting three pieces of wood—the resources disappear from the environment. (2) *Altering Internal States*. Actions taken by the agent can also change the agent itself, including

updating memories, forming new plans, acquiring novel knowledge, and more. For example, in Generative Agents [20], memory streams are updated after performing actions within the system. Similarly, SayCan [79] enables agents to take actions to update understandings of the environment. (3) *Triggering New Actions*. In task completion processes, one action often leads to subsequent actions. For example, in Voyager [38], once the agent has gathered the necessary resources, it triggers the construction of buildings.

2.2 Agent Capability Acquisition

In the sections above, we focus mainly on how to design the agent architecture to better harness the capabilities of LLMs to enabling them to accomplish tasks akin to human performance. The architecture functions as the “hardware” of an agent. However, relying solely on the hardware is insufficient for achieving effective task performance. This is because the agent may lack the necessary task-specific capabilities, skills, and experiences, which can be regarded as “software” resources. In order to equip the agent with these resources, various strategies have been devised. Generally, we categorize these strategies into two classes based on whether they require fine-tuning of the LLMs. Below, we introduce each category in detail.

Capability Acquisition with Fine-tuning: A direct approach to enhance agent capabilities for task completion is to fine-tune the model using task-specific datasets. These datasets can be constructed from human annotations, LLM-generated content, or real-world applications. We discuss these methods in detail below.

- *Fine-tuning with Human Annotated Datasets.* To fine-tune the agent, utilizing human annotated datasets is a versatile approach that can be employed

Table 1 For the profile module, we use ①, ② and ③ to represent the handcrafting method, LLM-generation method, and dataset alignment method, respectively. For the memory module, we focus on the implementation strategies for memory operation and memory structure. For memory operation, we use ① and ② to indicate that the model only has read/write operations and has read/write/reflection operations, respectively. For memory structure, we use ① and ② to represent unified and hybrid memories, respectively. For the planning module, we use ① and ② to represent planning w/o feedback and w/ feedback, respectively. For the action module, we use ① and ② to represent that the model does not use tools and use tools, respectively. For the agent capability acquisition (CA) strategy, we use ① and ② to represent the methods with and without fine-tuning, respectively. “-” indicates that the corresponding content is not explicitly discussed in the paper.

Model	Profile	Memory		Planning	Action	CA	Time
		Operation	Structure				
WebGPT [67]	-	-	-	-	②	①	12/2021
SayCan [79]	-	-	-	①	①	②	04/2022
MRKL [73]	-	-	-	①	②	-	05/2022
Inner Monologue [62]	-	-	-	②	①	②	07/2022
Social Simulacra [80]	②	-	-	-	①	-	08/2022
ReAct [60]	-	-	-	②	②	①	10/2022
MALLM [42]	-	①	②	-	①	-	01/2023
DEPS [33]	-	-	-	②	①	②	02/2023
Toolformer [15]	-	-	-	①	②	①	02/2023
Reflexion [12]	-	②	②	②	①	②	03/2023
CAMEL [81]	① ②	-	-	②	①	-	03/2023
API-Bank [70]	-	-	-	②	②	②	04/2023
ViperGPT [75]	-	-	-	-	②	-	03/2023
HuggingGPT [13]	-	①	①	①	②	-	03/2023
Generative Agents [20]	①	②	②	②	①	-	04/2023
LLM+P [58]	-	-	-	①	①	-	04/2023
ChemCrow [76]	-	-	-	②	②	-	04/2023
OpenAGI [74]	-	-	-	②	②	①	04/2023
AutoGPT [82]	-	①	②	②	②	②	04/2023
SCM [35]	-	②	②	-	①	-	04/2023
Socially Alignment [83]	-	①	②	-	①	①	05/2023
GITM [16]	-	②	②	②	①	②	05/2023
Voyager [38]	-	②	②	②	①	②	05/2023
Introspective Tips [84]	-	-	-	②	①	②	05/2023
RET-LLM [41]	-	①	②	-	①	①	05/2023
ChatDB [40]	-	①	②	②	②	-	06/2023
S ³ [78]	③	②	②	-	①	-	07/2023
ChatDev [18]	①	②	②	②	①	②	07/2023
ToolLLM [14]	-	-	-	②	②	①	07/2023
MemoryBank [39]	-	②	②	-	①	-	07/2023
MetaGPT [23]	①	②	②	②	②	-	08/2023

in various application scenarios. In this approach, researchers first design annotation tasks and then recruit workers to complete them. For example, in CoH [85], the authors aim to align LLMs with human values and preferences. Different from the other models, where the human feedback is leveraged in a simple and symbolic manner, this method converts the human feedback into detailed comparison information in the form of natural languages.

The LLMs are directly fine-tuned based on these natural language datasets. In RET-LLM [41], in order to better convert natural languages into structured memory information, the authors fine-tune LLMs based on a human constructed dataset, where each sample is a “triplet-natural language” pair. In WebShop [86], the authors collect 1.18 million real-world products from amazon.com, and put them onto a simulated e-commerce website, which con-

tains several carefully designed human shopping scenarios. Based on this website, the authors recruit 13 workers to collect a real-human behavior dataset. At last, three methods based on heuristic rules, imitation learning and reinforcement learning are trained based on this dataset. Although the authors do not fine-tune LLM-based agents, we believe that the dataset proposed in this paper holds immense potential to enhance the capabilities of agents in the field of web shopping. In EduChat [87], the authors aim to enhance the educational functions of LLMs, such as open-domain question answering, essay assessment, Socratic teaching, and emotional support. They fine-tune LLMs based on human annotated datasets that cover various educational scenarios and tasks.

- *Fine-tuning with LLM Generated Datasets.*

Building human-annotated datasets typically requires recruiting people, which can be costly, especially when dealing with large-scale annotation tasks. Considering that LLMs can achieve human-like capabilities in a wide range of tasks, a natural idea is using LLMs to accomplish the annotation task. While the datasets produced from this method can be not as perfect as the human annotated ones, it is much cheaper, and can be leveraged to generate more samples. For example, in ToolBench [14], to enhance the tool-using capability of open-source LLMs, the authors collect 16,464 real-world APIs spanning 49 categories from the RapidAPI Hub. They used these APIs to prompt ChatGPT to generate diverse instructions, covering both single-tool and multi-tool scenarios. Based on the obtained dataset, the authors fine-tune LLaMA [9], and obtain significant performance improvement in terms of tool using. In [83], to empower the agent with social capability, the authors design a sandbox, and deploy multiple agents to interact with each other. Given a social

question, the central agent first generates initial responses. Then, it shares the responses to its nearby agents for collecting their feedback. Based on the feedback as well as its detailed explanations, the central agent revise its initial responses to make them more consistent with social norms. In this process, the authors collect a large amount of agent social interaction data, which is then leveraged to fine-tune the LLMs.

- *Fine-tuning with Real-world Datasets.* In addition to building datasets based on human or LLM annotations, directly using real-world datasets to fine-tune the agent is also a common strategy. For example, in MIND2WEB [88], the authors collect a large amount of real-world datasets to enhance the agent capability in the web domain. In contrast to prior studies, the dataset presented in this paper encompasses diverse tasks, real-world scenarios, and comprehensive user interaction patterns. Specifically, the authors collect over 2,000 open-ended tasks from 137 real-world websites spanning 31 domains. Using this dataset, the authors fine-tune LLMs to enhance their performance on web-related tasks such as movie discovery and ticket booking. Similarly, in SQL-PaLM [89], researchers fine-tune PaLM-2 using cross-domain, large-scale text-to-SQL datasets, including Spider and BIRD. The resulting model achieves notable performance improvements on text-to-SQL tasks, demonstrating the effectiveness of real-world datasets for domain-specific applications.

Capability Acquisition without Fine-tuning: In the era of tradition machine learning, the model capability is mainly acquired by learning from datasets, where the knowledge is encoded into the model parameters. In the era of LLMs, the model capability can be acquired either by training/fine-tuning the model parameters or designing delicate prompts

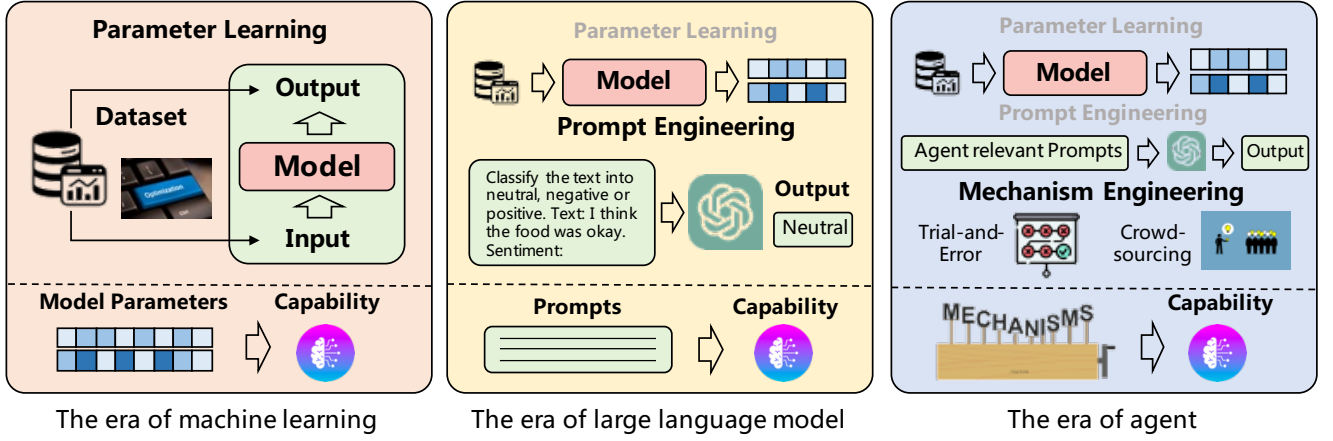


Fig. 4 Illustration of transitions in strategies for acquiring model capabilities.

(i.e., prompt engineering). In prompt engineering, one needs to write valuable information into the prompts to enhance the model capability or unleash existing LLM capabilities. In the era of agents, the model capability can be acquired based on three strategies: (1) model fine-tuning, (2) prompt engineering and (3) designing proper agent evolution mechanisms (we called it as *mechanism engineering*). Mechanism engineering is a broad concept that involves developing specialized modules, introducing novel working rules, and other strategies to enhance agent capabilities. For clearly understanding the transitions of model capability acquisition strategies, we illustrate them in Figure 4. In the following, we detail prompting engineering and mechanism engineering.

- **Prompting Engineering.** Due to the strong language comprehension capabilities, people can directly interact with LLMs using natural languages. This introduces a novel strategy for enhancing agent capabilities, that is, one can describe the desired capability using natural language and then use it as prompts to influence LLM actions. For example, in CoT [45], in order to empower the agent with the capability for complex task reasoning, the authors present the intermediate reasoning steps as few-shot

examples in the prompt. Similar techniques are also used in CoT-SC [51] and ToT [52]. In RLP [30], the authors aim to enhance an agent’s self-awareness in conversations by prompting LLMs with the agent’s beliefs about both its own and the listeners’ mental states. This approach results in more engaging and adaptive utterances. Furthermore, the incorporation of the target mental states of listeners allows the agent to formulate more strategic plans. Retroformer [90] presents a retrospective model that enables the agent to generate reflections on its past failures. The reflections are integrated into the prompt of LLMs to guide the agent’s future actions. Additionally, this model utilizes reinforcement learning to iteratively improve the retrospective model, thereby refining the LLM prompt.

- **Mechanism Engineering.** Unlike model fine-tuning and prompt engineering, mechanism engineering is a unique strategy to enhance agent capability. In the following, we present several representative methods of mechanism engineering.

- (1) **Trial-and-error.** In this method, the agent first performs an action, and subsequently, a pre-defined critic is invoked to judge the action. If the action is deemed unsatisfactory, then the agent reacts by incorporating the critic’s feedback. For example, in

RAH [91], the agent serves as a user assistant in recommender systems. One of the agent's crucial roles is to simulate human behavior and generate responses on behalf of the user. To fulfill this objective, the agent first generates a predicted response and then compares it with the real human feedback. If the predicted response and the real human feedback differ, the critic generates failure information, which is subsequently incorporated into the agent's next action. Similarly, in DEPS [33], the agent first designs a plan to accomplish a given task. In the plan execution process, if an action fails, the explainer generates specific details explaining the cause of the failure. This information is then incorporated by the agent to redesign the plan. In RoCo [92], the agent first proposes a sub-task plan and a path of 3D waypoints for each robot in a multi-robot collaboration task. The plan and waypoints are then validated by a set of environment checks, such as collision detection and inverse kinematics. If any of the checks fail, the feedback is appended to each agent's prompt and another round of dialog begins. The agents use LLMs to discuss and improve their plan and waypoints until they pass all validations. PREFER [93] extends this idea by leveraging LLMs to generate detailed feedback when the agent underperforms, enabling iterative refinement and performance improvement.

(2) *Crowd-sourcing*. In [94], the authors design a debating mechanism that leverages the wisdom of crowds to enhance agent capabilities. To begin with, different agents provide separate responses to a given question. If their responses are not consistent, they will be prompted to incorporate the solutions from other agents and provide an updated response. This iterative process continues until reaching a final consensus answer. In this method, the capability of each agent is enhanced by understand-

ing and incorporating the other agents' opinions.

(3) *Experience Accumulation*. In GITM [16], the agent does not know how to solve a task in the beginning. Then, it makes explorations, and once it has successfully accomplished a task, the actions used in this task are stored into the agent memory. In the future, if the agent encounters a similar task, then the relevant memories are extracted to complete the current task. In this process, the improved agent capability comes from the specially designed memory accumulation and utilization mechanisms. Voyager [38] introduces a skill library, where executable codes for specific skills are refined through interactions with the environment, enabling efficient task execution over time. In AppAgent [95], the agent is designed to interact with apps in a manner akin to human users, learning through both autonomous exploration and observation of human demonstrations. Throughout this process, it constructs a knowledge base, which serves as a reference for performing intricate tasks across various applications on a mobile phone. In MemPrompt [96], the users are requested to provide feedback in natural language regarding the problem-solving intentions of the agent, and this feedback is stored in memory. When the agent encounters similar tasks, it attempts to retrieve related memories to generate more suitable responses.

(4) *Self-driven Evolution*. This method allows agents to autonomously improve through self-directed learning and feedback mechanisms. LMA3 [97] enables the agent to autonomously set goals for itself, and gradually improve its capability by exploring the environment and receiving feedback from a reward function. Following this mechanism, the agent can acquire knowledge and develop capabilities according to its own preferences. SALLMS [98] integrates advanced LLMs like GPT-4 into a multi-agent system, agents can adapt and perform